

Clase 4 - Carga de datos

Joselina Davyt

Espacio de trabajo y organización de proyectos de ciencia de datos

- Una buena organización de archivos es la base de cualquier proyecto profesional
- Estructura de directorios: mantener una estructura clara e independiente del sistema:
 - data/raw/ (datos originales e inmutables)
 - data/processed/ (limpios)
 - notebooks/ o src/ o scripts (código)
 - outputs/ o resultados (reportes y modelos)

Rutas

- ¿qué son las rutas?: definen en qué carpeta y archivos estoy trabajando en el proyecto de ciencia de datos
- Preferible usar rutas relativas y no absolutas, porque las rutas absolutas impiden la reproducibilidad de los proyectos.
- Ruta relativa: “data/raw/”
- Ruta absoluta: “C:/Users/...”
- Las rutas son texto (=string/=character) para los lenguajes de programación
- Existen paquetes y funciones que interpretan la ruta como tal

Trabajar con rutas

R Python

```
1 getwd() # al ejecutarlo me dice en dónde está parado el entorno ahora
```

```
[1] "C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-datos/clases/clase4"
```

```
1 # sirve para fijar la ruta del entorno o espacio de trabajo, genera rutas absolutas no relativas
2 setwd("C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-datos/clases/clase4")
```

Antes, si estas trabajando dentro de RStudio ejecuta (dentro de VS no es necesario)

```
1 reticulate::py_require(c("pandas", "numpy", "matplotlib"))
```

En python permite trabajar con rutas absolutas y relativas:

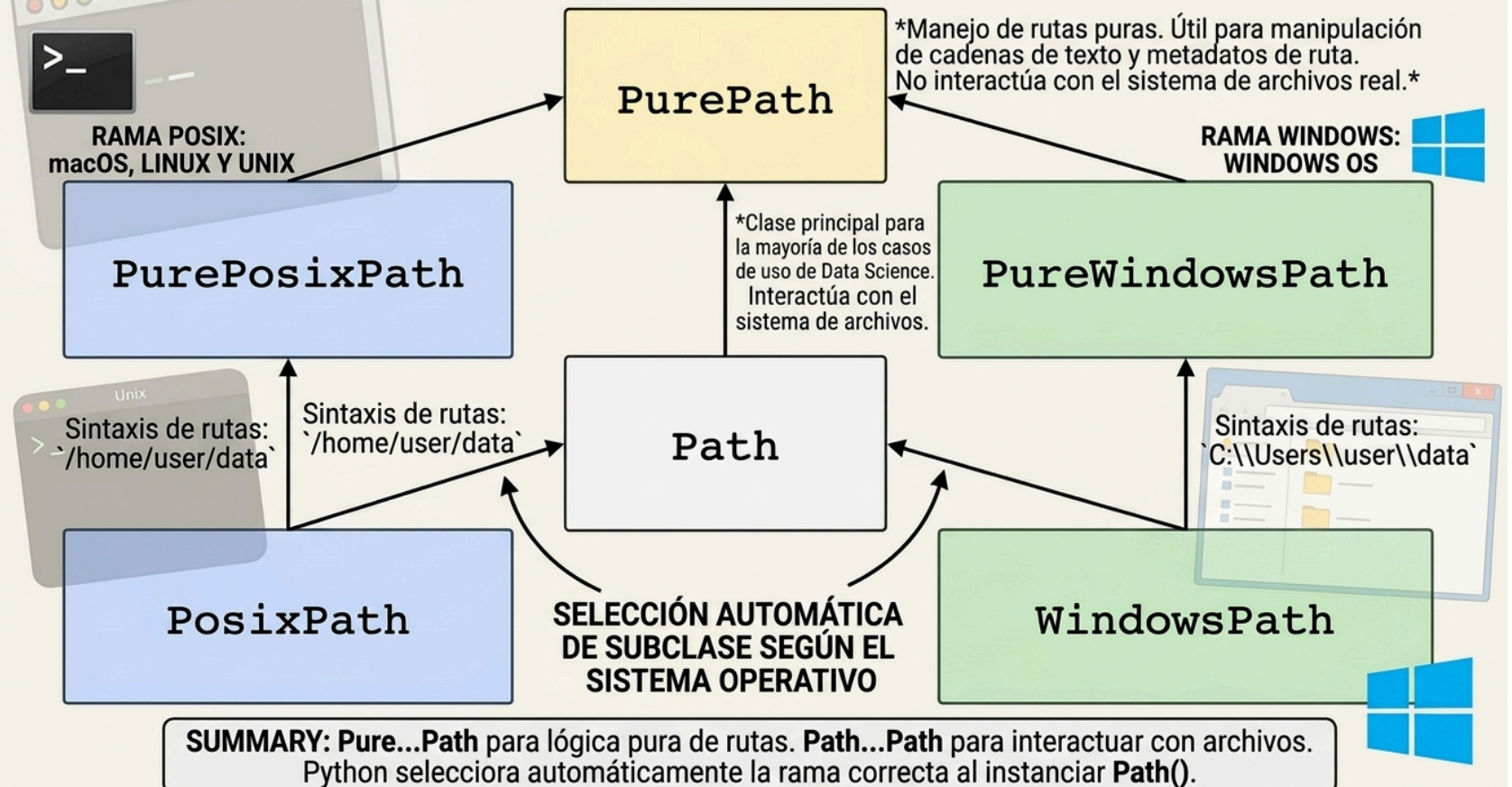
```
1 import os
2
3 # es el equivalente a getwd() de R
4 os.getcwd()
5 # Subir un nivel en el árbol de carpetas (equivalente a 'cd ..')
6 os.chdir("../")
7 os.getcwd()
```

```
1 # Entrar a una subcarpeta desde el directorio actual
2 os.chdir("clases/clase4")
3 # ruta absoluta:
4 os.chdir("C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-datos/clases/clase3")
```

Trabajar con rutas: el enfoque moderno

Usar con paquetes para trabajar desde la raíz del proyecto y sus subcarpetas.

ENTENDIENDO LA JERARQUÍA DE CLASES DE PATHLIB EN PYTHON



Trabajar con rutas: el enfoque moderno

R Python

```
1 # 1 - instalar paquete
2 #install.packages("here")
3 # 2- cargar paquete
4 library(here)
```

```
1 # 1 - instalar paquete
2 #pip install pathlib
3 # 2 - cargar paquete
4 from pathlib import Path
5 # listado de subdirectorios
6 p = Path('.')
```

Carga de csv

Fuente: Catalogo de datos abiertos Uy

https://catalogodatos.gub.uy/dataset/

Conjuntos de Datos

Compartir

Filtros

Buscar conjuntos de datos...

Organizaciones

MIDES

1.875

Intendencia de Montevideo

154

ANCAP

60

Correo Uruguayo

43

OSE

28

Mostrar Más Organizaciones

Ordenar por

Relevancia

2.702 conjuntos de datos encontrados

Intendencia de Montevideo

Estación de Transferencia de Residuos Sólidos Urbanos

La Estación de Transferencia de Residuos, ubicada en Ruta 102 y Cno Manuel M. Fortet, funciona como punto intermedio para la descarga de residuos entre circuitos de la zona...

CSV

Última actualización: 1 de septiembre de 2026, 5:18 (UTC-03:00)

Publicador: Intendencia de Montevideo

Categorías: No hay categorías asociadas a este conjunto de datos

Etiquetas: Harvest , ambiente , limpieza , residuos sólidos

localhost:7855/#/title-slide

11/23

Leer csv

R Python

```
1 library(here)
2 library(readr)
3
4 # 1. Construir la ruta relativa hacia el archivo CSV
5 # Estructura esperada: tu_proyecto/data/raw/datos.csv
6 archivo <- "cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"
7 ruta_csv <- here(archivo)
8 ruta_csv <- here("clases", "clase4", archivo)
9 ruta_csv
```

```
[1] "C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-
datos/clases/clase4/cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"
```

```
1 # 2. Verificar que el archivo exista
2 if (!file.exists(ruta_csv)) {
3   stop(paste("No se encontró el archivo en:", ruta_csv))
4 }
5
6 # 3. Cargar el archivo CSV
7 df <- read_csv(ruta_csv)
8
9 # 4. Inspección inicial de los datos
10 cat("--- Archivo cargado exitosamente desde:", basename(ruta_csv), "---\n")
```

--- Archivo cargado exitosamente desde: cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv ---

```
1 print(spec(df)) # Muestra el tipo de dato asignado a cada columna
```

```
cols(
  pesada = col_double(),
  fecha_dia = col_character(),
  fecha = col_character(),
  matricula_letra = col_character(),
  tipo_residuo = col_double(),
  tipo_residuo_desc = col_character(),
  subtipo_residuo = col_double(),
  subtipo_residuo_desc = col_character(),
  peso_netto = col_double(),
  peso_bruto = col_double(),
  tara = col_double(),
  balanza = col_double(),
  medio_transp = col_double(),
  medio_transp_desc = col_character(),
  trayecto = col_logical(),
  trayecto_desc = col_logical(),
  tipo_recorrido = col_logical(),
  tipo_recorrido_desc = col_logical(),
  tipo_volcado_desc = col_character(),
  nomenclatura_circuito = col_logical(),
  municipio = col_logical(),
```

```
1 head(df)
```

```
# A tibble: 6 × 41
```

	pesada	fecha_dia	fecha	matricula_letra	tipo_residuo	tipo_residuo_desc
	<dbl>	<chr>	<chr>	<chr>	<dbl>	<chr>
1	3996241	30/11/23 00:00	30/11/2...	SIM	1	TIPO DOMICILIARIO

```

2 3996243 30/11/23 00:00 30/11/2... SIM      1 TIPO DOMICILIARIO
3 3996253 30/11/23 00:00 30/11/2... SIM      1 TIPO DOMICILIARIO
4 3996276 30/11/23 00:00 30/11/2... SIM      1 TIPO DOMICILIARIO
5 3996298 30/11/23 00:00 30/11/2... SIM      1 TIPO DOMICILIARIO
6 3996303 30/11/23 00:00 30/11/2... SIM      1 TIPO DOMICILIARIO
# i 35 more variables: subtipo_residuo <dbl>, subtipo_residuo_desc <chr>,
# peso_netto <dbl>, peso_bruto <dbl>, tara <dbl>, balanza <dbl>,
# medio_transp <dbl>, medio_transp_desc <chr>, trayecto <lgl>,
# trayecto_desc <lgl>, tipo_recorrido <lgl>, tipo_recorrido_desc <lgl>,
# tipo_volcado_desc <chr>, nomenclatura_circuito <lgl>, municipio <lgl>,
# lugar_salida <lgl>, lugar_salida_desc <lgl>, permiso <lgl>,
# nro_permiso <dbl>, permiso_id <dbl>, tipo_permiso <dbl>, ...

```

```

1 from pathlib import Path
2 import pandas as pd
3
4 # 0. nombre archivo
5 archivo = "cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"
6
7 # 1. Definir la ruta raíz del proyecto o del directorio de trabajo actual
8 BASE_DIR = Path.cwd()
9
10 # 2. Construir la ruta relativa de forma segura utilizando el operador /
11 # Estructura esperada: tu_proyecto/data/raw/datos.csv
12 ruta_csv = BASE_DIR / "clases" / "clase4" / archivo
13
14 # 3. Verificar que el archivo realmente existe antes de cargarlo
15 if not ruta_csv.exists():
16     raise FileNotFoundError(f"No se encontró el archivo en: {ruta_csv}")
17
18 # 4. Cargar el archivo CSV
19 df = pd.read_csv(ruta_csv)
20
21 # 5. Inspección inicial de los datos
22 print(f"--- Archivo cargado exitosamente desde: {ruta_csv.name} ---")
23 print(df.info())

```

```
24 print("\nPrimeras 5 filas:")  
25 print(df.head())
```

Data profiling/Inspección de datos/Reconocimiento de datos

Actividades esenciales:

- Evaluar el tamaño del archivo y las necesidades de procesamiento
- Evaluar si el caso a analizar está completo con los datos obtenidos: filas (unidades de observación) y columnas (características o variables)
- Confirmar el tipo de datos obtenidos
- Identificar el delimitador (separación entre columnas)
- Revisar inconsistencias en los nombres de las variables
- Esta etapa debe culminar con un código definitivo de carga de datos (tipo de conexión, tipo de archivos, tipos de datos, completitud)

Unidad de observación

- La unidad de observación refiere al objeto de estudio
- Permite identificar las columnas IDs por las cuales se podrían unir datos
- Ejemplos video juegos:
 - Nivel Acción. **Fila:** Evento atómico (*Acción dentro del juego*), ejemplo `event_id`.
 - Nivel Partida. **Fila:** Partida (*Match*) o Jugador-Partida (*Player-Match*), ejemplo campos: `match_id`, `player_id`, `kills`, `deaths`, `win_bool`.
 - Nivel jugador. **Fila:** Un Jugador único (*Perfil Agregado*), por ejemplo calcular predicción de abandono, retención a 7/30 días y gasto acumulado. **Campo:** `player_id`
 - Nivel título. **Fila:** Un videojuego o título (*App Level*). **Objetivo:** Estudio de catálogo, estrategia de precios y competencia por género. **Campos:** `app_id`, `title`, `genre`, `price`, `peak_concurrent_users`.

Problemas de identificación de datos e integración

- Verificar consistencia de formato de IDs, ¿por qué? porque son las variables mediante las cuales se unen distintas tablas de datos.
- Verificar duplicados
- Distintas fechas de generación de registros, por ejemplo: la solicitud de un tratamiento se realiza en diciembre de 2025 y se realiza en 2026
- Duplicados: más de un registro por IDs, ¿es necesario o se quiere mantener
- Solución: comenzar con una lista de los NO SÉ.

Data profiling: identificador

R Python

```
1 # dimensiones y estructura de los datos
2 str(df) # 1222 filas y 41 columnas
```

```
spc_tbl_ [1,222 x 41] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ pesada          : num [1:1222] 4e+06 4e+06 4e+06 4e+06 4e+06 ...
 $ fecha_dia       : chr [1:1222] "30/11/23 00:00" "30/11/23 00:00" "30/11/23 00:00" "30/11/23 00:00" ...
 $ fecha          : chr [1:1222] "30/11/23 08:28" "30/11/23 08:30" "30/11/23 08:36" "30/11/23 08:56" ...
 $ matricula_letra : chr [1:1222] "SIM" "SIM" "SIM" "SIM" ...
 $ tipo_residuo    : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
 $ tipo_residuo_desc : chr [1:1222] "TIPO DOMICILIARIO" "TIPO DOMICILIARIO" "TIPO DOMICILIARIO" "TIPO
DOMICILIARIO" ...
 $ subtipo_residuo : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
 $ subtipo_residuo_desc : chr [1:1222] "ASIMILABLE A DOMICILIARIO" "ASIMILABLE A DOMICILIARIO" "ASIMILABLE A
DOMICILIARIO" "ASIMILABLE A DOMICILIARIO" ...
 $ peso_netto      : num [1:1222] 11590 8660 6090 7920 9740 ...
 $ peso_bruto      : num [1:1222] 28450 26070 25020 24720 26890 ...
 $ tara            : num [1:1222] 16860 17410 18930 16800 17150 ...
 $ balanza         : num [1:1222] 5 5 5 5 5 5 5 5 5 5 ...
 $ medio_transp    : num [1:1222] 1 1 50 1 1 50 1 1 1 50 ...
 $ medio_transp_desc : chr [1:1222] "COMPACTADOR LEVANTE LAT." "COMPACTADOR LEVANTE LAT." "COMPACTADOR LAT.CAJA
DESM" "COMPACTADOR LEVANTE LAT." ...
 $ trayecto        : logi [1:1222] NA NA NA NA NA NA ...
 $ trayecto_desc   : logi [1:1222] NA NA NA NA NA NA ...
 $ tipo_recorrido  : logi [1:1222] NA NA NA NA NA NA ...
 $ tipo_recorrido_desc : logi [1:1222] NA NA NA NA NA NA ...
```

```
1 # filas únicas: detectando la columna que contiene el ID
2 length(unique(df$matricula_letra)) #¿cuál es la columna identificador?
```

[1] 12

```
1 import pandas as pd
2
3 # 1. Dimensiones y estructura de los datos (Equivalente a str(df))
4 df.info()
5 # Nota: df.shape te da exactamente las dimensiones (filas, columnas) -> (1222, 41)
6
7 # 2. Filas únicas: detectando la columna que contiene el ID (Equivalente a length(unique(...)))
8 df["matricula_letra"].nunique()
```

Data profiling 2: valores faltantes

R Python

```
1 colSums(is.na(df)) # Conteo de NA por columna
```

pesada	fecha_dia	fecha
0	0	0
matricula_letra	tipo_residuo	tipo_residuo_desc
0	0	0
subtipo_residuo	subtipo_residuo_desc	peso_netto
0	0	0
peso_bruto	tara	balanza
0	0	0
medio_transp	medio_transp_desc	trayecto
0	0	1222
trayecto_desc	tipo_recorrido	tipo_recorrido_desc
1222	1222	1222
tipo_volcado_desc	nomenclatura_circuito	municipio
0	1222	1222
lugar_salida	lugar_salida_desc	permiso
1222	1222	1222
nro_permiso	permiso_id	tipo_permiso
828	828	828
tipo_permiso_desc	peso_authorized	destino_volcado
828	1222	0
destino_volcado_desc	tipo_pesada	tipo_pesada_desc
0	0	0

```
1 apply(df, function(x) sum(x == "" | x == "N/A", na.rm = TRUE)) # Falsos nulos
```

pesada	fecha_dia	fecha
0	0	0
matricula_letra	tipo_residuo	tipo_residuo_desc
0	0	0
subtipo_residuo	subtipo_residuo_desc	peso_netto
0	0	0
peso_bruto	tara	balanza
0	0	0
medio_transp	medio_transp_desc	trayecto
0	0	0
trayecto_desc	tipo_recorrido	tipo_recorrido_desc
0	0	0
tipo_volcado_desc	nomenclatura_circuito	municipio
0	0	0
lugar_salida	lugar_salida_desc	permiso
0	0	0
nro_permiso	permiso_id	tipo_permiso
0	0	0
tipo_permiso_desc	peso_authorized	destino_volcado
0	0	0
destino_volcado_desc	tipo_pesada	tipo_pesada_desc
0	0	0

```
1 df.isnull().sum() # Conteo de NA por columna
2 (df == "").sum() # Falsos nulos (cadenas vacías)
```

Data profiling 3: duplicados

R Python

```
1 sum(duplicated(df)) # Cantidad de filas exactamente iguales
```

```
[1] 0
```

```
1 df.duplicated().sum() # Cantidad de filas exactamente iguales
```



Speaker notes